Guia tecnica del codigo, flujo de ejecucion y paths configurables

Proyecto EMG Prosthesis TD3

25 de marzo de 2026

1. Objetivo de esta guia

Esta guia describe el codigo activo del proyecto, que modulos se usan, para que sirven, como se conectan entre si y que rutas o parametros deben cambiarse cuando el proyecto se mueve de maquina, pasa de simulacion a hardware o se quiere evaluar un checkpoint concreto.

El foco no es resumir experimentos, sino dejar claro **como funciona el repo hoy**. La configuracion valida actual sigue basada en:

- observacion `markov52`,
- reward `trackingMseActionRateReward`,
- TD3 feedforward,
- remapeo cuantizado del actuador.

2. Que contiene el proyecto

La carpeta principal relevante es `EMG_Prosthesis_TD3/matlab_code`. Dentro de ella, el flujo moderno del proyecto se reparte asi:

Cuadro 1: Mapa funcional de carpetas y archivos

Ruta	Funcion principal
`config/configurables.m`	Punto central de configuracion: modo train/test, dataset, reward, TD3, paths, hardware y opciones de guardado.
`src/trainInterface.m`	Entry point operativo para entrenamiento y evaluacion.
`agents/load_agent.m`	Fabrica de agentes; decide que agente crear segun `agent_id`.
`agents/agentTd3.m`	Construccion del actor, criticos y opciones TD3.
`src/@Env`	Entorno RL: estado, paso temporal, logging, guardado por episodio y remapeo de accion.
`src/reward functions`	Reward actual y variantes historicas o experimentales.
`src/@SimController`	Simulador de la protesis y capa de control de motores.
`src/runCheckpointAudit.m`	Auditoria automatizada de checkpoints.
`src/runCheckpointTest.m`	Test directo de un checkpoint con simulacion y plots.
`data/datasets`	Datasets EMG+glove usados en simulacion.
`docs/td3_training_report`	Documentacion tecnica, reportes y guias.

3. Pipeline real del codigo

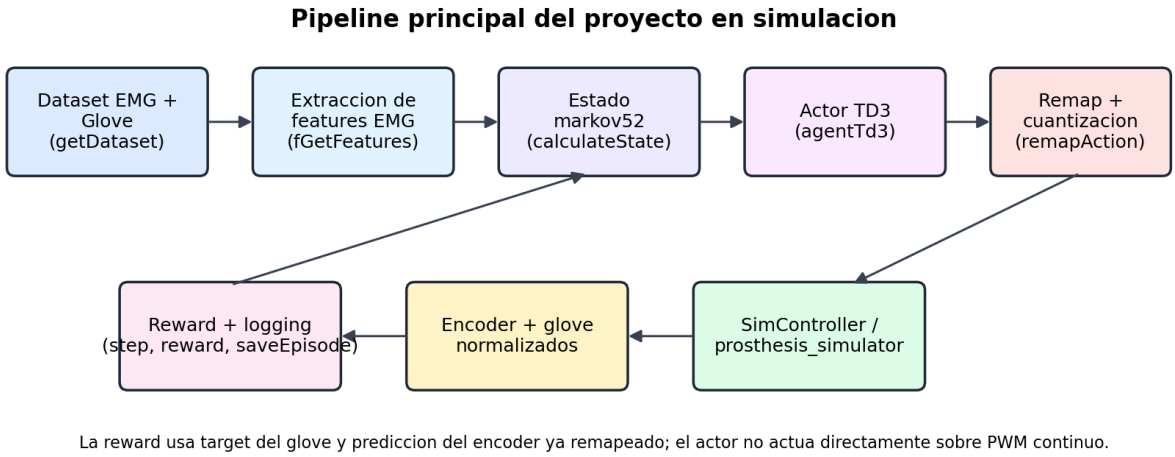

Figura 1: Pipeline principal del proyecto en simulacion.

En ejecucion normal, el flujo es:

1. MATLAB entra en `trainInterface.m`.
2. `configurables()` crea el struct de configuracion activo.
3. Si hay entrenamiento nuevo, `load_agent()` crea un TD3 desde cero.
4. Si hay evaluacion o resume training, se carga un checkpoint y se reaplican overrides mutables.
5. `Env` recibe dataset prerecorded o hardware real.
6. En cada `step`, la accion del actor se remapea a la accion efectiva del actuador.
7. El simulador o controlador aplica PWM, lee encoder y glove, reconstruye el estado y calcula la reward.
8. El entorno guarda logs detallados por paso y por episodio.

4. Archivos clave y para que sirve cada uno

Cuadro 2: Archivos que conviene conocer primero

Archivo	Tipo	Responsabilidad
`config/configurables.m`	Configuracion	Define parametros del experimento y construye `RLtrainingOptions` o `simOpts`.
`src/trainInterface.m`	Launcher	Orquesta carga del agente, carga del dataset, creacion de entorno y ejecucion de `train` o `sim`.
`agents/agentTd3.m`	Modelo RL	Construye actor, criticos y opciones TD3.
`src/@Env/calculateState.m`	Estado	Convierte EMG y encoder a observacion `legacy44`, `markov52` o `stackedEmg132`.
`src/@Env/remapActionForActuator.m`	Accion efectiva	Aplica umbral y cuantizacion antes de enviar PWM.
`src/@Env/step.m`	Dinamica del entorno	Aplica accion, avanza tiempo, lee sensores, calcula reward y registra logs.
`src/reward functions/trackingMseActionRateReward.m`	Reward actual	Penaliza tracking MSE, magnitud de accion y cambio de accion.
`src/@SimController/prosthesis_simulator.m`	Simulador	Genera la dinamica de la protesis. Aqui se corrigio el fallback cuando `ws` estaba vacio.
`src/runCheckpointAudit.m`	Infraestructura	Lanza auditorias reproducibles de checkpoints.
`src/runCheckpointTest.m`	Evaluacion	Ejecuta una simulacion batch sobre un checkpoint concreto.

5. Lineas de codigo clave

5.1. `configurables.m`: configuracion global

Estas lineas son las mas importantes para entender que experimento se esta corriendo:

```
60: params.trainingMaxEpisodes = 12000;
61: params.trainingSaveAgentEvery = 100;
62: params.trainingPlots = "training-progress";

85: params.run_training = true;
93: params.newTraining = true;
102: params.agentFile = "";

146: params.dataset_folder = '.\data\datasets\Denis Dataset\';
150: params.comUNO = "COM4";
151: params.comGlove = "COM3";

157: params.rewardType = 'trackingMseActionRateReward';
163: params.rewardActionWeight = 0.01;
166: params.rewardDeltaActionWeight = 0.05;

180: params.speeds = 255 * [1, 1, 1, 1];
182: params.actionCommandActivationThreshold = 0.05;
183: params.actionCommandLevels = [0 64 96 128 160 192 224 255];
```

Mas abajo, el archivo cierra la definicion de observacion y runtime:

```
265: params.observationVariant = "markov52";
267: params.stateLength = params.numEMGFeatures + 3 * numel(params.minAction);

379: params.RLtrainingOptions = rlTrainingOptions(...)
388: params.simOpts = rlSimulationOptions(...)
```

5.2. `trainInterface.m`: donde empieza de verdad

```
57: if configs.newTraining
60:     observationInfo = Env.defineObservationInfo();
63:     agent = load_agent(observationInfo, actionInfo, agent_id, param_name, param_value
    );
65: else
67:     if strlength(string(configs.agentFile)) == 0
73:     vars = who('-file', configs.agentFile);
85:     if configs.run_training
86:         [agent, loadedAgentOverrides] = applyLoadedAgentTrainingOverrides(agent,
    configs);
```

Y la bifurcacion final entre entrenamiento y evaluacion es:

```
142: if configs.run_training
144:     opts = configs.RLtrainingOptions;
148:     trainingInfo = train(agent, env, opts);
149: else
151:     simpOpts = configs.simOpts;
156:     trainingInfo = sim(agent, env, simpOpts);
```

5.3. `agentTd3.m`: donde queda definido TD3

```
10: if isfield(td3, "useRecurrent") && td3.useRecurrent
13: else
14:     [actor, critics] = buildFeedforwardTd3Networks(...)
18: agentOptions = buildTd3AgentOptions(td3);
19: agent = rlTD3Agent(actor, critics, agentOptions);
```

En la configuracion valida actual, `useRecurrent = false`, asi que la ruta activa es la feedforward.

5.4. `remapActionForActuator.m`: aqui vive la discontinuidad importante

```
24: if magnitude < this.actionCommandActivationThreshold
25:     continue
28: targetPwm = magnitude * maxPwm;
29: [~, idx] = min(abs(nonZeroLevels - targetPwm));
30: pwmMagnitude = nonZeroLevels(idx);
32: appliedPwm(i) = sign(actionValue) * pwmMagnitude;
33: effectiveAction(i) = appliedPwm(i) / maxPwm;
```

Esta funcion explica por que el actor no controla una planta continua pura: hay umbral y cuantizacion antes del actuador.

5.5. `step.m`: la transicion del entorno

```
23: [effectiveAction, appliedPwm] = this.remapActionForActuator(action);
29: completed = this.prosthesis.sendAllSpeed(...)

46: % Advance episode time before reading the prosthesis...
48: this.episodeTic.toc(this.c);

100: [this.State, currentEncoderNorm] = this.calculateState(emg, motorData);
106: [reward, rewardVector, rewardInfo] = this.reward_function(this, effectiveAction, [])
    ;
113: this.trackingMseLog(this.c) = rewardInfo.trackingMse;
```

```
119: this.saturationFractionLog(this.c) = rewardInfo.saturationFraction;
139: if isDone && this.flagSaveTraining
140:     this.saveEpisode();
```

Las lineas 46–48 son especialmente importantes: ahi quedo corregido el desfase temporal para que la accion afecte el paso actual.

5.6. `trackingMseActionRateReward.m`: la reward vigente

```
13: err = pred - target;
14: trackingMse = mean(err.^2);
16: actionL2 = mean(action.^2);
19: deltaAction = action - prevAction;
20: deltaActionL2 = mean(deltaAction.^2);
25: rewardVector = -(err.^2 + actionPenaltyVector + deltaActionPenaltyVector);
26: reward = mean(rewardVector);
35: "saturationFraction", mean(abs(action) >= 0.95)
```

La reward actual no empuja directamente por progreso o direccion, sino por error cuadratico medio y regularizacion de accion.

5.7. `prosthesis_simulator.m`: el bug importante ya corregido

```
94: ws = sim_params.(sp_txt).(dir).(m_txt).ws;
106: useCurveFallback = ws_len == 0;
108: % fit_C2.mat currently stores empty ws trajectories
110: ws = curve;
111: ws_len = curve_len;
```

Sin este fallback, el simulador podia quedarse inmovil aunque hubiese PWM no nulo. Por eso las corridas previas al fix no se consideran benchmark valido.

6. Paths y parametros que si debes cambiar

6.1. Regla general

Si trabajas en **simulacion** y mantienes la estructura actual del repo, casi todo ya esta en rutas relativas. Los cambios obligatorios aparecen solo cuando:

- mueves el dataset fuera del repo,
- quieres evaluar o reanudar desde un checkpoint concreto,
- cambias la carpeta de salida,
- o activas hardware real.

6.2. Tabla de paths configurables

Cuadro 3: Paths y parametros que conviene revisar

Parametro o ruta	Archivo y lineas	Cuando cambiarlo	Observacion practica
`params.dataset_folder`	`configurables.m: 132-147`	Solo si el dataset no esta en `matlab_code/data/datasets/Denis Dataset`	En un clon limpio del repo no hace falta tocarlo.

Parametro o ruta	Archivo y lineas	Cuando cambiarlo	Observacion practica
`params.dataset`	`configurables.m: 135-145`	Si quieres entrenar o evaluar con otro subconjunto de sujetos	Puede ser string unico o celda de nombres.
`params.agent File`	`configurables.m: 99-106`	Obligatorio al evaluar o reanudar entrenamiento	Debe apuntar al `AgentXXXX.mat` deseado. Si queda vacio, `trainInterface` falla a proposito.
`params.agents_ directory`	`configurables.m: 197-200`	Si quieres guardar resultados en otra raiz	Por defecto guarda en `../../Agentes/ trainedAgentsProtesisTest/...` .
`params.comUNO`	`configurables.m: 149-151`	Solo hardware real	Puerto serie de la protesis. No afecta simulacion.
`params.comGlove`	`configurables.m: 149-151`	Solo hardware real con glove real	Puerto serie del guante. No afecta simulacion prerecorded.
`agentFile` de `runProsthesis.m`	`runProsthesis.m: 33-40`	Solo si usas el script legacy de hardware	No tiene default valido. Debe ponerse a mano.
`defaultCheckpoint Path()`	`runCheckpointTest.m: 42-49`	Solo si llamas a `runCheckpoint Test()` sin argumentos	Ese fallback es historico; lo correcto es pasar la ruta explicitamente.
`cd(.../ matlab_code)`	No es archivo, es requisito operativo	Siempre	Los comandos de train/test/audit asumen que MATLAB esta en `EMG_Prosthesis_TD3/ matlab_code` .

6.3. Paths que normalmente no debes tocar

- `.\config\normValues.mat`: ya se carga relativo en `configurables.m:209-213`.
- `.\src`, `.\agents`, `.\lib`: los `addpath` ya se resuelven desde `trainInterface.m:49-52`.
- rutas internas de auditoria: las herramientas nuevas ya trabajan repo-relativas si se les pasa una carpeta de experimento o checkpoints explicitos.

7. Comandos utiles

7.1. Sesion limpia

```
cd('C:/ruta/al/repo/EMG_Prosthesis_TD3/matlab_code')
clearConfigurablesOverride()
```

7.2. Entrenamiento

```
cd('C:/ruta/al/repo/EMG_Prosthesis_TD3/matlab_code')
clearConfigurablesOverride()
trainInterface('td3','','')
```

7.3. Evaluacion o test de un checkpoint

Antes de llamar a `trainInterface`, deja en `configurables.m`:

- `params.run_training = false`
- `params.newTraining = false`
- `params.agentFile = ".../AgentXXXX.mat"`

O bien usa el helper directo:

```
cd('C:/ruta/al/repo/EMG_Prosthesis_TD3/matlab_code')
clearConfigurablesOverride()
trainingInfo = runCheckpointTest("C:/ruta/al/Agent7250.mat", 50, true);
```

7.4. Auditoria de checkpoints

```
cd('C:/ruta/al/repo/EMG_Prosthesis_TD3/matlab_code')
clearConfigurablesOverride()
results = runCheckpointAudit(50, 200, 3, struct( ...
    'experimentDir', 'C:/ruta/a/la/corrida', ...
    'samplingPolicy', struct('mode', 'tail_every_k_last_n', 'k', 50, 'n', 12)));
```

8. Que mirar antes de tocar codigo

1. **Si cambia el comportamiento del agente**, mira primero `configurables.m`.
2. **Si el actor parece mandar cosas raras**, mira `remapActionForActuator.m`.
3. **Si el tracking deja de tener sentido**, revisa `step.m`, `calculateState.m` y la reward.
4. **Si la simulacion parece inmovil**, revisa `prosthesis_simulator.m`.
5. **Si un checkpoint no carga bien**, revisa `trainInterface.m` y `applyLoadedAgentTrainingOverrides.m`.

9. Que no conviene tocar sin una razon fuerte

- `observationVariant`, `rewardType` y arquitectura del agente al mismo tiempo.
- `actionCommandActivationThreshold` y `actionCommandLevels` sin volver a auditar el efecto en control.
- `runCheckpointTest()` sin pasar checkpoint explicito si quieres resultados reproducibles.
- scripts legacy de hardware si el objetivo actual es simulacion.

10. Cierre

La idea de esta guia es que puedas abrir el repo y responder rapido cuatro preguntas:

1. donde arranca el flujo,
2. donde se construye el TD3,
3. donde se convierte la accion continua en accion efectiva,
4. y que rutas o parametros debes cambiar segun el escenario.

Si se mantiene la estructura actual del repo y se trabaja en simulacion, los cambios manuales necesarios son pocos. Los dos puntos que mas se suelen tocar son `params.agentFile` y, solo si mueves datos o hardware, `dataset_folder`, `comUNO` y `comGlove`.